

Lessons Learned — Analisi del Rischio

Idea generale

L'analisi del rischio serve a ragionare in modo ordinato su ciò che potrebbe andare storto.

Non significa essere pessimisti.

Significa fare una cosa molto più intelligente:

prima immagino i problemi possibili,
poi decido cosa fare per evitarli o limitarne i danni.

In informatica, nei progetti, nella sicurezza, nelle aziende e perfino nella vita quotidiana, il rischio è sempre presente.

La differenza tra lavorare "a caso" e lavorare in modo professionale è che il rischio viene identificato, valutato e gestito.

Cos'è un rischio

Un rischio è un evento possibile che, se si verifica, può avere conseguenze negative.

Esempi:

- il server si blocca
- un file importante viene cancellato
- un utente inserisce dati sbagliati
- una password viene rubata
- una consegna non viene completata in tempo
- un software contiene bug gravi
- una persona chiave del team si ammala
- un backup non funziona quando serve

La parola importante è:

possibile

Un rischio non è ancora successo.

Se è già successo, non è più un rischio: è un problema.

Rischio vs problema

Questa distinzione è fondamentale.

Un rischio riguarda qualcosa che potrebbe accadere.

Un problema riguarda qualcosa che è già accaduto.

Esempio:

Rischio:
il database potrebbe non essere raggiungibile.

Problema:
il database non è raggiungibile.

Nel primo caso possiamo prepararci.

Nel secondo caso dobbiamo intervenire.

Da cosa è composto un rischio

Un rischio di solito si valuta usando due elementi principali:

probabilità
impatto

La probabilità indica quanto è possibile che il rischio si verifichi.

L'impatto indica quanto sarebbe grave il danno se si verificasse.

Formula semplificata:

$\text{rischio} = \text{probabilità} \times \text{impatto}$

Questa formula non serve sempre a fare matematica precisa.

Serve soprattutto a ragionare.

Esempio semplice

Rischio:

perdere il codice sorgente di un progetto

Probabilità:

media

Perché può succedere se si lavora solo in locale.

Impatto:

alto

Perché si può perdere molto lavoro.

Contromisura:

usare Git e fare push su un repository remoto

Qui vediamo il punto chiave:

analizzare un rischio serve a decidere un'azione concreta.

La matrice del rischio

Una matrice del rischio aiuta a classificare i rischi combinando probabilità e impatto.

Esempio semplificato:

Probabilità	Impatto basso	Impatto medio	Impatto alto
Bassa	Rischio basso	Rischio basso	Rischio medio
Media	Rischio basso	Rischio medio	Rischio alto
Alta	Rischio medio	Rischio alto	Rischio critico

La matrice aiuta a capire dove concentrare l'attenzione.

Non tutti i rischi meritano lo stesso sforzo.

Priorità dei rischi

Un errore comune è trattare tutti i rischi allo stesso modo.

In realtà bisogna dare priorità.

Esempio:

Rischio A:
un bottone ha un colore poco leggibile.

Rischio B:
gli utenti possono perdere i dati inseriti.

Rischio C:
chiunque può accedere all'area amministrativa.

Non sono rischi uguali.

Il rischio C è probabilmente il più grave.

Il rischio B è molto importante.

Il rischio A può essere fastidioso, ma non è critico.

Le fasi dell'analisi del rischio

Un processo semplice di analisi del rischio può essere diviso così:

1. identificazione
2. valutazione
3. prioritizzazione
4. trattamento
5. monitoraggio

1. Identificazione del rischio

Qui ci chiediamo:

cosa potrebbe andare storto?

Esempi di domande utili:

- cosa succede se il sistema si blocca?

- cosa succede se l'utente inserisce dati sbagliati?
- cosa succede se manca internet?
- cosa succede se un file viene cancellato?
- cosa succede se una persona non consegna in tempo?
- cosa succede se un dato viene letto da chi non dovrebbe?
- cosa succede se una funzione viene usata in modo non previsto?

Questa fase richiede attenzione e immaginazione controllata.

Non bisogna inventare catastrofi assurde, ma nemmeno ignorare problemi prevedibili.

2. Valutazione del rischio

Dopo aver trovato un rischio, bisogna valutarlo.

Domande utili:

```
quanto è probabile?  
quanto sarebbe grave?  
quanto sarebbe facile accorgersene?  
quanto sarebbe difficile rimediare?
```

Esempio:

```
Rischio:  
un utente inserisce testo invece di un numero.  
  
Probabilità:  
alta.  
  
Impatto:  
medio.  
  
Contromisura:  
validare l'input con TryParse.
```

Questo esempio è perfetto per C# console.

Il rischio è prevedibile.

La soluzione è tecnica e semplice.

3. Prioritizzazione

Non tutti i rischi si risolvono subito.

La prioritizzazione serve a decidere cosa affrontare prima.

Di solito si parte dai rischi con:

alta probabilità
alto impatto

Esempio:

Alta probabilità + alto impatto = rischio urgente.
Bassa probabilità + basso impatto = rischio secondario.

4. Trattamento del rischio

Una volta valutato il rischio, bisogna decidere cosa fare.

Le strategie principali sono:

- evitare
- ridurre
- trasferire
- accettare

Evitare il rischio

Evitare un rischio significa cambiare strada per eliminarlo.

Esempio:

Non voglio rischiare di perdere dati importanti salvati solo sul PC.

Soluzione:

Non salvo solo in locale.
Uso GitHub, backup o cloud.

Ridurre il rischio

Ridurre un rischio significa abbassarne probabilità o impatto.

Esempio:

Rischio:
l'utente inserisce un voto non valido.

Soluzione:

controllo che il voto sia tra 0 e 10.

Il rischio non sparisce al 100%, ma viene gestito.

Trasferire il rischio

Trasferire un rischio significa spostarne la responsabilità o l'impatto su qualcun altro.

Esempio:

un'azienda usa un servizio cloud invece di gestire fisicamente i server.

Questo non elimina il rischio, ma lo sposta in parte sul fornitore.

Accettare il rischio

Accettare un rischio significa decidere consapevolmente di non intervenire.

Questo può avere senso se:

- il rischio è molto basso
- il costo per evitarlo è troppo alto
- l'impatto è minimo

Esempio:

Rischio:
un piccolo refuso in un commento del codice.

Non è bello, ma magari non è urgente.

Il registro dei rischi

In un progetto reale, i rischi possono essere raccolti in una tabella chiamata registro dei rischi.

Esempio:

ID	Rischio	Probabilità	Impatto	Livello	Azione
R1	Input non valido	Alta	Media	Alto	Validare input
R2	Perdita codice	Media	Alta	Alto	Usare Git remoto
R3	Bug nel calcolo media	Media	Media	Medio	Testare funzione
R4	Ritardo consegna	Media	Alta	Alto	Pianificare step piccoli

Il registro serve a non tenere tutto "a memoria".

La memoria umana è simpatica, ma spesso mente come un compilatore senza warning.

Analisi del rischio in programmazione

In programmazione l'analisi del rischio è ovunque.

Esempi pratici:

Input utente

Rischio:

l'utente inserisce dati non validi.

Soluzione:

validazione con TryParse, controlli semantici, messaggi chiari.

Array

Rischio:

accedere a un indice non valido.

Soluzione:

controllare Length, evitare indici fuori intervallo.

Calcoli

Rischio:

dividere per zero.

Soluzione:

controllare che il divisore sia diverso da zero.

File

Rischio:

il file non esiste o non è leggibile.

Soluzione:

controllare l'esistenza del file e gestire gli errori.

Database

Rischio:

query errate, dati corrotti, accessi non autorizzati.

Soluzione:

validazione, backup, permessi, transazioni.

Analisi del rischio e sicurezza informatica

Nella sicurezza informatica il rischio è centrale.

Esempi:

Rischio	Possibile conseguenza	Contromisura
Password debole	Account compromesso	Password robuste, MFA
Dati non cifrati	Furto informazioni	Cifratura
Permessi troppo ampi	Accesso non autorizzato	Principio del minimo privilegio
Software non aggiornato	Vulnerabilità sfruttabili	Aggiornamenti
Backup assenti	Perdita dati	Backup periodici e testati

Una regola molto importante è:

non basta fare il backup.
bisogna anche verificare che il backup funzioni.

Un backup mai testato è una coperta che forse è un tovagliolo.

Analisi del rischio nei progetti

Nei progetti software i rischi non sono solo tecnici.

Esistono anche rischi organizzativi.

Esempi:

- requisiti poco chiari
- tempi sottostimati
- comunicazione debole
- dipendenza da una sola persona
- scarsa documentazione
- modifiche continue
- test insufficienti

Esempio pratico:

Rischio:
il cliente cambia idea a metà progetto.

Possibile conseguenza:

ritardi, aumento costi, confusione.

Contromisure:

```
requisiti scritti
approvazioni intermedie
versioni incrementali
comunicazione regolare
```

Analisi del rischio e metodo “prima piccolo, poi grande”

Una buona strategia per ridurre il rischio è lavorare a piccoli passi.

In programmazione significa:

```
non scrivere tutto il programma in una volta.
```

Meglio:

1. creo menu
2. provo menu
3. aggiungo inserimento voti
4. provo inserimento
5. aggiungo media
6. provo media

Questo riduce il rischio di arrivare alla fine con un programma enorme che non funziona e non si capisce dove sia il problema.

La scimmietta saggia non costruisce il castello partendo dal tetto.

Rischio residuo

Anche dopo aver applicato contromisure, può restare un rischio.

Questo si chiama rischio residuo.

Esempio:

```
Uso TryParse per validare un numero.
```

Il rischio di input non numerico è gestito.

Ma potrebbero restare altri rischi:

numero fuori intervallo
utente che preme invio vuoto
logica sbagliata dopo la conversione

L'analisi del rischio non dice mai:

ora siamo immortali.

Dice:

abbiamo ridotto il rischio a un livello accettabile.

Rischio, errore ed eccezione

In informatica possiamo distinguere:

rischio
errore
eccezione

Rischio:

qualcosa potrebbe andare storto.

Errore:

qualcosa è stato fatto male o ha prodotto un risultato sbagliato.

Eccezione:

il programma incontra una situazione anomala durante l'esecuzione.

Esempio:

Rischio:
l'utente inserisce abc invece di un numero.

Errore:
il programma prova a convertirlo come numero senza controllo.

Eccezione:
`int.Parse("abc")` genera un errore di runtime.

Contromisura:

`int.TryParse(...)`

Analisi del rischio applicata all'esercizio voti

Prendiamo un esercizio console sui voti di una classe.

Rischi possibili:

Rischio	Probabilità	Impatto	Contromisura
Utente inserisce testo invece di numero	Alta	Media	<code>TryParse</code>
Utente inserisce voto fuori 0-10	Alta	Media	controllo intervallo
Array pieno	Media	Media	limite massimo e contatore
Calcolo media senza voti	Media	Alta	controllare <code>count > 0</code>
Minimo/massimo senza voti	Media	Alta	controllare <code>count > 0</code>
Zeri non inseriti trattati come voti	Alta	Alta	usare contatore voti reali
Scelta menu non valida	Alta	Bassa	messaggio e ripetizione menu

Questo è un esempio perfetto di analisi del rischio concreta.

Non è teoria astratta.

Serve a scrivere codice migliore.

Concetto chiave: il rischio guida il design

Una cosa importante:

l'analisi del rischio non viene dopo il codice.
viene prima e durante il codice.

Se so che un utente può inserire dati sbagliati, progetto subito una funzione di lettura sicura.

Se so che un array ha capienza massima, progetto subito un contatore.

Se so che non posso calcolare una media senza voti, controllo subito il caso vuoto.

Il rischio, quindi, aiuta a decidere la struttura del programma.

Esempio di funzione guidata dal rischio

Rischio:

l'utente inserisce un voto non numerico o fuori intervallo.

Possibile funzione:

```
int ReadGrade()
{
    int grade;

    while (!int.TryParse(Console.ReadLine(), out grade) || grade < 0 ||
grade > 10)
    {
        Console.Write("Voto non valido. Inserisci un valore tra 0 e 10: ");
    }

    return grade;
}
```

Qui il codice nasce da un rischio preciso.

Non stiamo aggiungendo controlli "per bellezza".

Stiamo proteggendo il programma da un caso prevedibile.

Domande guida da farsi sempre

Quando scriviamo un programma, possiamo chiederci:

cosa succede se l'utente sbaglia input?
cosa succede se non ci sono dati?
cosa succede se ci sono troppi dati?
cosa succede se un valore è fuori intervallo?
cosa succede se un file manca?
cosa succede se una funzione riceve parametri non validi?
cosa succede se il programma viene usato in modo diverso da come immaginavo?

Queste domande trasformano un programmatore “che scrive codice” in un programmatore “che progetta software”.

Errori comuni

Errore 1:

`pensare che il rischio sia solo una cosa da aziende grandi.`

Falso.

Il rischio esiste anche in un esercizio console.

Errore 2:

`controllare solo che il programma compili.`

Compilare non significa funzionare bene.

Errore 3:

`fidarsi dell'utente.`

Mai fidarsi dell'utente.

Non perché l'utente sia cattivo.

Perché l'utente è creativo.

A volte troppo.

Errore 4:

`ignorare il caso vuoto.`

Molti programmi si rompono quando non ci sono dati.

Esempio:

calcolare la media di zero voti.

Errore 5:

confondere capienza e contenuto reale.

Un array da 10 posti non significa che contiene davvero 10 voti validi.

Serve un contatore.

Morale finale

L'analisi del rischio è un modo professionale di pensare.

Non serve a dire:

andrà tutto male.

Serve a dire:

se qualcosa può andare male, io mi preparo.

In programmazione questo porta a:

- input più sicuri
- programmi più robusti
- meno errori runtime
- codice più leggibile
- scelte progettuali migliori
- meno panico quando qualcosa non va

Un buon programmatore non scrive solo codice che funziona quando tutto va bene.

Scriva codice che si comporta decentemente anche quando qualcosa va storto.

Questa è la differenza tra:

codice fragile

e:

software pensato